seminar

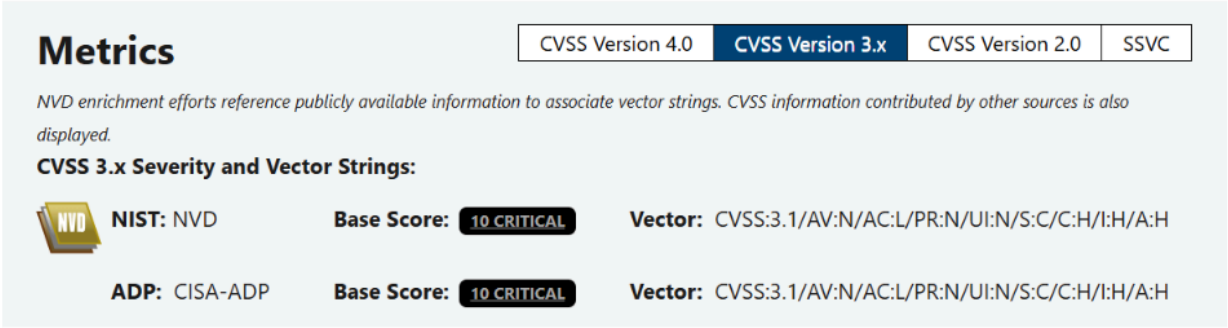
Integer Overflow - Phân tích hiện tượng giá trị số vượt quá phạm vi biểu diễn, dẫn đến sai lệch trong kiểm tra, tính toán kích thước hoặc cấp phát bộ nhớ. Làm rõ cách lỗi số học có thể tạo ra lỗ hổng bảo mật.

Mỗi nhóm lựa chọn (qua hình thức giao đề tài) một chủ đề về khai thác lỗ hổng phần mềm và thực hiện trình bày trong thời lượng tối đa 20–30 phút, bao gồm lý thuyết, phân tích kỹ thuật và demo:

- Trình bày khái niệm, nguyên lý và điều kiện phát sinh của lỗ hổng.
- Xác định và giải thích mã CWE tương ứng.
- Nhận diện vị trí lỗ hổng trong mã nguồn hoặc binary và phân tích nguyên nhân gây ra lỗ hổng.
- ***Phân tích ít nhất một CVE thực tế có liên quan, bao gồm sản phẩm bị ảnh hưởng, nguyên nhân, tác động, điều kiện khai thác và biện pháp khắc phục.***
- ***Giải thích chuỗi khai thác và mối liên hệ giữa lỗ hổng, CWE, CVE và kỹ thuật khai thác.***
- ***Phần CVE chỉ yêu cầu nghiên cứu và phân tích thông tin; không bắt buộc reproduce hoặc tái hiện việc khai thác CVE.***
- Thực hiện demo giải 01 challenge CTF thuộc mảng Pwnable trong môi trường được cấp phép, cô lập và an toàn.
- Đề xuất biện pháp khắc phục hoặc phòng ngừa.
- Phân tích các điều kiện, cơ chế bảo vệ và giới hạn ảnh hưởng đến khả năng khai thác thành công.
- Chuẩn bị slide, tài liệu tham khảo và phương án dự phòng cho demo khi cần thiết.
- Việc khai thác chỉ được thực hiện trên challenge CTF, máy ảo, container hoặc hệ thống được cho phép; nghiêm cấm kiểm thử trên hệ thống thật hoặc ngoài phạm vi cấp phép.

CVE-2020-0796:

Định nghĩa: A remote code execution vulnerability exists in the way that the Microsoft Server Message Block 3.1.1 (SMBv3) protocol handles certain requests, aka 'Windows SMBv3 Client/Server Remote Code Execution Vulnerability'.



Sản phẩm bị ảnh hưởng:

- Windows 10 Version 1903 for 32-bit Systems
- Windows 10 Version 1903 for x64-based Systems
- Windows 10 Version 1903 for ARM64-based Systems
- Windows Server, version 1903 (Server Core installation)
- Windows 10 Version 1909 for 32-bit Systems
- Windows 10 Version 1909 for x64-based Systems
- Windows 10 Version 1909 for ARM64-based Systems
- Windows Server, version 1909 (Server Core installation)

`Nói chung:`

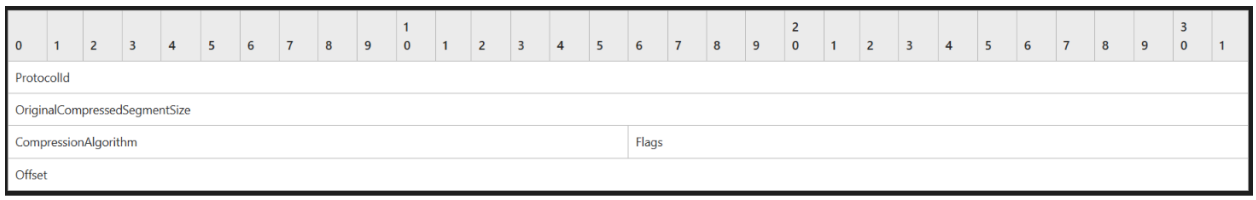
CVE-2020-0796, còn gọi là SMBGhost, là lỗ hổng trong quá trình xử lý thông điệp có nén của SMB 3.1.1 trên các phiên bản Windows bị ảnh hưởng. Lỗ hổng có thể dẫn đến lỗi bộ nhớ và thực thi mã từ xa không cần xác thực.

Phần phân tích dưới đây tập trung vào đường xử lý phía SMB server trong driver `srv2.sys` của bản Windows chưa được vá.

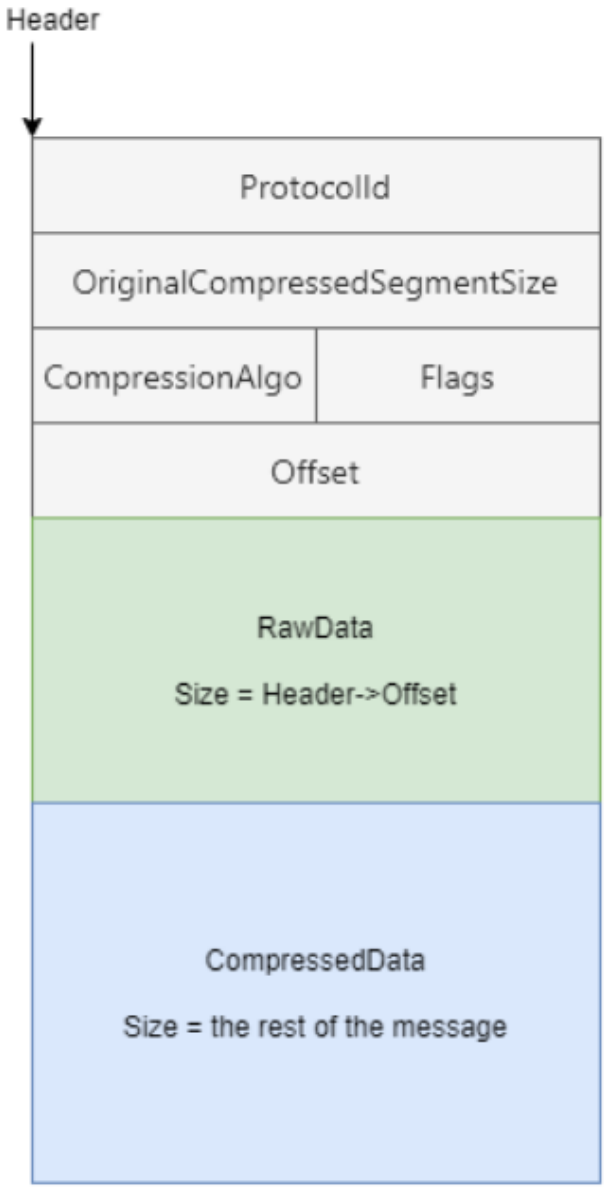
Giải thích:

Câú trúc thông điệp SMB có nén – dạng unchained

- The SMB2 COMPRESSION_TRANSFORM_HEADER is used by the client or server when sending compressed messages.



- **OriginalCompressedSegmentSize (4 bytes):** The size, in bytes, of the uncompressed data segment.
- **Offset (4 bytes):** The offset, in bytes, from the end of this structure to the start of compressed data segment
- Như vậy, gói data được nén sẽ có dạng sau:



Luồng xử lý thông điệp SMB có nén phía server:

```
  Srv2ReceiveHandler
            |
            |
            v
Srv2DecompressMessageAsync
            |
            |
            v
```

```
    Srv2DecompressData  ------->  SrvNetAllocateBuffer
            |
            |
            v
 SmbCompressionDecompress
        |
        |
        v
    memcpy
```

Đầu tiên hàm `Srv2ReceiveHandler` được gọi để nhận một smb data packet và gọi một hàm tương ứng với giao thức `ProtocolId`. Nếu `PrococolId` = 0x424D53FC, nó sẽ gọi hàm `Srv2DecompressMessageAsync`, hàm này sẽ tiến hành gọi hàm `Srv2DecompressData` để giải nén data packet. Hàm `Srv2DecompressData` sẽ gọi hàm `SrvNetAllocateBuffer` để cấp phát một `Alloc` dùng để lưu data sau khi giải nén, kế đến nó gọi hàm `SmbCompressionDecompress` để tiến hành giải nén data packet, sau cùng sẽ gọi hàm `memcpy`. Như vậy toàn bộ quá trình Decompress sẽ có các bước chính sau:

1. Allocate
2. Decompress
3. Copy

Hàm Srv2DecompressData

```
__int64 __fastcall Srv2DecompressData(__int64 _smb_packet)
{
  __int64 smb_packet; // rdi
  __int64 _header; // rax
  SMB_V2_COMPRESSION_TRANSFORM_HEADER v3; // xmm0
  AAA smb_header_compress; // xmm0_8
  unsigned int CompressionAlgorithm; // ebp
  __int64 __alloc_buffer; // rax
  __int64 __allocated_buffer; // rbx
  int PayloadSize; // eax
  SMB_V2_COMPRESSION_TRANSFORM_HEADER Header; // [rsp+30h] [r
```

```
bp-28h]
  int UncompressedSize; // [rsp+60h] [rbp+8h]

  UncompressedSize = 0;
  smb_packet = _smb_packet;
  _header = *(_QWORD *)(_smb_packet + 0xF0);

  // Basic size checks
  if ( *(_DWORD *)(_header + 0x24) < sizeof(SMB_V2_COMPRESSIO
N_TRANSFORM_HEADER) )
    return 0xC000090Bi64;

  v3 = *(SMB_V2_COMPRESSION_TRANSFORM_HEADER *)*(_QWORD *)(_h
eader + 0x18);
  Header = v3;

  // Check the compression algo used is the same one as the o
ne negotiated during NEGOTIATE_PACKET sequence
  *(__m128i *)&smb_header_compress.Algo = _mm_srli_si128(
                                            (__m128i)v3,
                                            offsetof(SMB_V2_C
OMPRESSION_TRANSFORM_HEADER, CompressionAlgorithm));
  CompressionAlgorithm = *(_DWORD *)(*(_QWORD *)(*(_QWORD *)
(_smb_packet + 80) + 496i64) + 140i64);
  if ( CompressionAlgorithm != (unsigned __int16)smb_header_c
ompress.Algo )
    return 0xC00000BBi64;

  // Nani ?? oO
  __alloc_buffer = SrvNetAllocateBuffer(
        (unsigned int)(
            Header.OriginalCompressedSegmentSize + smb_header
_compress.OffsetOrLength),
        0i64
  );
```

```
  __allocated_buffer = __alloc_buffer;
  if ( !__alloc_buffer )
    return 0xC000009Ai64;

  // Decompress data in newly allocated buffer and check the
uncompressed size is equal to the one filled out in Header.Or
iginalCompressedSegmentSize
  if ( (int)SmbCompressionDecompress(
              CompressionAlgorithm,
              (BYTE *)(*(_QWORD *)(*(_QWORD *)(smb_packet + 2
40) + 24i64) + (unsigned int)Header.OffsetOrLength

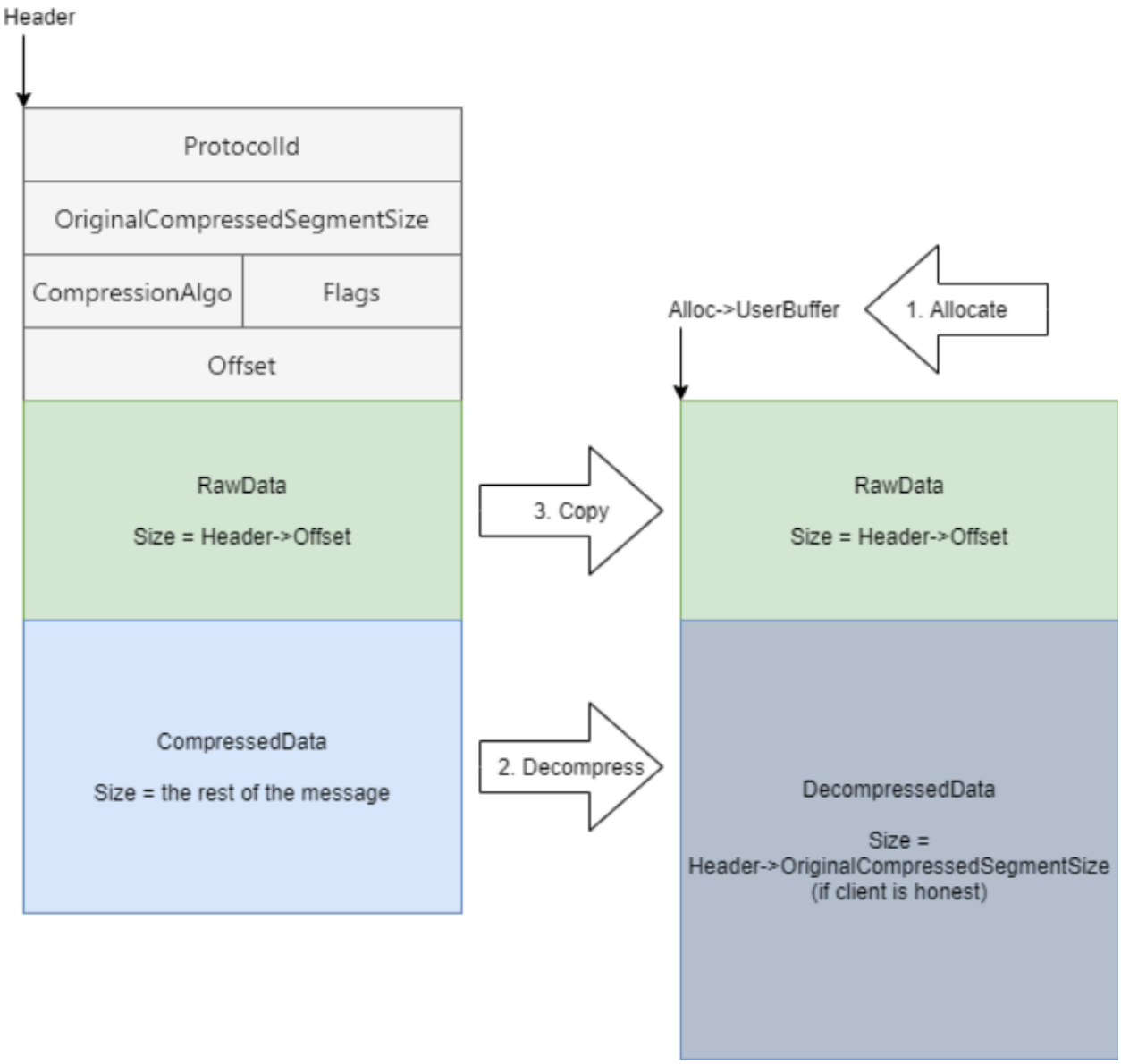
+ 0x10i64),
              *(_DWORD *)(*(_QWORD *)(smb_packet + 240) + 36i
64) - Header.OffsetOrLength - 0x10,
              (BYTE *)((unsigned int)Header.OffsetOrLength +
*(_QWORD *)(__alloc_buffer + 0x18)),
              Header.OriginalCompressedSegmentSize,
              &UncompressedSize) < 0
    || (PayloadSize = UncompressedSize, UncompressedSize != H
eader.OriginalCompressedSegmentSize) )
  {
    SrvNetFreeBuffer(__allocated_buffer);
    return 0xC000090Bi64;
  }

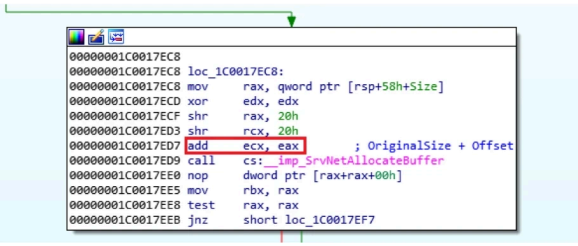
  // Copy optional payload
  if ( Header.OffsetOrLength )
  {
    memmove(
      *(void **)(__allocated_buffer + 0x18),
      (const void *)(*(_QWORD *)(*(_QWORD *)(smb_packet + 24
0) + 24i64) + 0x10i64),
      (unsigned int)Header.OffsetOrLength);
    PayloadSize = UncompressedSize;
  }
```

```
  *(_DWORD *)(__allocated_buffer + 36) = Header.OffsetOrLengt
h + PayloadSize;
  Srv2ReplaceReceiveBuffer(smb_packet, __allocated_buffer);
  return 0i64;
}
```

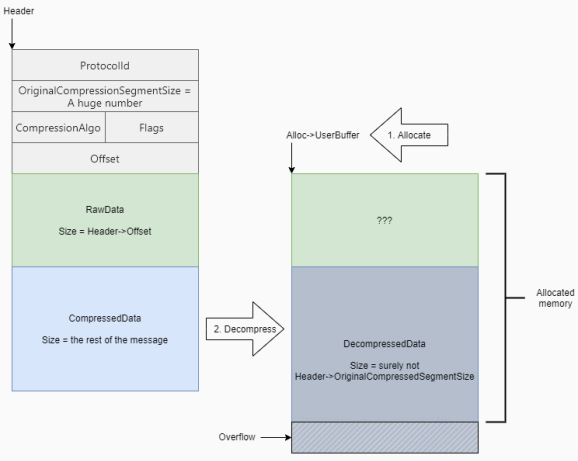
Phân tích hàm `Srv2DecompressData`, nhận thấy hàm nhận vào một packet data nén `COMPRESSION_TRANSFORM_HEADER` (Header), tiến hành cấp phát một vùng nhớ (Alloc) bằng hàm `SrvNetAllocateBuffer` với tham số là tổng `Header.OriginalCompressedSegmentSize + smb_header_compress.OffsetOrLength`, sau đó giải nén dữ liệu được nén và copy dữ liệu không được nén vào `Alloc->Buffer`.

```
 __alloc_buffer = SrvNetAllocateBuffer(
        (unsigned int)(
            Header.OriginalCompressedSegmentSize + smb_header
_compress.OffsetOrLength),
        0i64
  );
```





Lỗi integer overflow xảy ra khi `Srv2DecompressData` tính tổng `OriginalCompressedSegmentSize + Offset` bằng phép cộng 32 bit để xác định kích thước vùng đệm. Nếu tổng vượt quá giá trị lớn nhất của số nguyên không dấu 32 bit, kết quả bị cuộn về một giá trị nhỏ.



Ví dụ, tổng đúng của `0xFFFFFFFF + 0x10` là `0x1_0000000F`, nhưng kết quả phép cộng 32 bit chỉ còn `0x0000000F`, tương đương 15.

Tổng đã bị tràn được truyền vào tham số kích thước của `SrvNetAllocateBuffer`; tham số thứ hai là `NULL`. Dù tham số kích thước nhận giá trị 64 bit, việc mở rộng kết quả sau phép cộng không khôi phục phần đã mất. Vùng đệm được cấp phát theo kích thước sai, khiến các thao tác giải nén hoặc sao chép sau đó có thể ghi vượt giới hạn.

Điều kiện khai thác:

Các điều kiện để kích hoạt lỗi gồm:

- Hệ thống thuộc phiên bản Windows bị ảnh hưởng và chưa áp dụng bản vá.
- Thông điệp đi đến đường xử lý SMB 3.1.1 có hỗ trợ compression.
- Các trường kích thước trong header khiến phép cộng 32 bit bị tràn.
- Kết quả sai được dùng để cấp phát vùng đệm và chương trình tiếp tục xử lý dữ liệu.

Phía server có thể bị tấn công khi nhận kết nối SMB độc hại. Phía client có thể bị tấn công khi kết nối với SMB server độc hại. Lỗ hổng có thể bị khai thác không cần xác thực.

Việc kích hoạt lỗi có thể gây crash hoặc hỏng bộ nhớ. Để đạt thực thi mã từ xa, người tấn công còn phải kiểm soát tác động của lỗi bộ nhớ và vượt qua các cơ chế bảo vệ liên quan.

Biện pháp khắc phục:

- Cập nhật bản vá của Microsoft cho CVE-2020-0796: https://support.microsoft.com/en-au/servicing/os/windows-

10/2020/03/march-12-2020-kb4551762-os-builds-18362-720-and-18363-720-expired

- AFTER:

```
 unsigned int _v_allocation_size = 0;

  if (!NT_SUCCESS(RtlUlongAdd(Header.OriginalCompressedSegmentSize, smb_header_compress.OffsetOrLength, &_v_allocation_size)))
  {
    SEND_SOME_ETW_EVENT_FOR_TELEMETRY_AND_CATCHING_BAD_GUYS(&wpp_guid);
    goto ON_ERROR;
  }

  if (_v_allocation_size > another_smb_size_i_guess)
  {
    SEND_SOME_ETW_EVENT_FOR_TELEMETRY_AND_CATCHING_BAD_GUYS(&wpp_guid);
    goto ON_ERROR;
  }

  __alloc_buffer = SrvNetAllocateBuffer(
    _v_allocation_size,
    0i64
  );
  if ( !__alloc_buffer )
    return 0xC000009A;

  if (!NT_SUCCESS(RtlULongSub(_v_allocation_size, smb_header_compress.OffsetOrLength, &_v_uncompressed_size)))
  {
    SEND_SOME_ETW_EVENT_FOR_TELEMETRY_AND_CATCHING_BAD_GUYS(&wpp_guid);
    goto ON_ERROR;
```

```
  }

  if (!NT_SUCCESS(SmbCompressionDecompress(
              AlgoId,
              (BYTE *)(*(_QWORD *)(*(_QWORD *)(smb_packet
+ 240) + 24i64) + (unsigned int)Header.OffsetOrLength

+ 0x10i64),
              _v_uncompressed_size,
              Size.m128i_u32[3] + *(_QWORD *)(v10 + 24),
              Header.OriginalCompressedSegmentSize,
              &UncompressedSize)) < 0
    || (PayloadSize = UncompressedSize, UncompressedSize !
= Header.OriginalCompressedSegmentSize) )
```

Vấn đề	Cách phòng ngừa
`OriginalCompressedSegmentSize + Offset` bị tràn, dẫn đến cấp phát vùng đệm quá nhỏ	Kiểm tra phép cộng trước khi dùng kết quả để cấp phát; từ chối thông điệp nếu phép tính bị tràn.
`Offset` vượt quá dữ liệu thực nhận, khiến vị trí đọc nằm ngoài vùng nguồn	Kiểm tra thông điệp đủ chứa header 16 byte và `Offset` không vượt quá phần dữ liệu còn lại.
Phép tính độ dài nguồn bị underflow	Chỉ thực hiện phép trừ sau khi xác nhận các giới hạn hợp lệ.
Dữ liệu giải nén vượt vùng đích	Ràng buộc kích thước đầu ra theo dung lượng vùng đệm đã cấp phát.

3. Giảm nguy cơ khi chưa thể cập nhật

- **Tắt SMB compression trên server**: đặt `DisableCompression` thành `1` trong `HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters`. Biện pháp này không cần khởi động lại, nhưng **không bảo vệ SMB client**. CERT/CC
- **Hạn chế kết nối SMB qua TCP 445:** chặn kết nối từ Internet vào mạng nội bộ và từ mạng nội bộ ra Internet khi không cần thiết. Máy vẫn có thể bị tiếp cận từ những kết nối nội bộ được cho phép. CERT/CC
- **Dùng IPS để phát hiện hoặc chặn lưu lượng khai thác:** Fortinet công bố chữ ký liên quan đến lỗi Compression Transform Header. Đây là lớp bảo vệ bổ sung; máy vẫn cần được vá. Fortinet

Mối liên hệ tới CWE:

```
Phép cộng OriginalCompressedSegmentSize + Offset bị tràn
                          ↓
              CWE-190: Integer Overflow
                          ↓
            Tính sai kích thước vùng đệm
            CWE-131: Incorrect Calculation of Buffer Size
                          ↓
               Ghi ngoài vùng đích
            CWE-787: Out-of-bounds Write
```

```
Offset vượt giới hạn dữ liệu thực nhận
                          ↓
Vị trí bắt đầu đọc vượt vùng nguồn;
phép tính độ dài nguồn bị underflow
            CWE-191: Integer Underflow
                          ↓
               Đọc ngoài vùng nguồn
            CWE-125: Out-of-bounds Read
```

Trong ví dụ này, SMBGhost là lỗ hổng cụ thể và CVE-2020-0796 là mã định danh của nó. CWE giúp mô tả các dạng sai sót bên trong: phép cộng bị tràn thuộc CWE-190, việc tính sai kích thước vùng đệm thuộc CWE-131, và ghi ngoài vùng nhớ thuộc CWE-787.

Kỹ thuật khai thác là cách tận dụng những sai sót đó thông qua thông điệp SMB nén được tạo có chủ đích. Các giá trị trong header kích hoạt lỗi số học, khiến chương trình thiết lập vùng nhớ và giới hạn xử lý sai. Kết quả có thể là crash; để đạt thực thi mã, người tấn công còn phải khai thác lỗi bộ nhớ và vượt qua các cơ chế bảo vệ.